

Lab 4-INFS343

Classification

Classification is perhaps the most common of all data mining tasks. Classification analysis uses the historical data that have classes. Classification is a data mining task of predicting the value of a categorical variable (class). The objective of classification is to analyze data and automatically generate a model that can predict future behavior. The induced model consisting of generalizations over the past data (called training data) helps distinguish predefined classes. The hope is that the model can then be used to predict the actual future events.

Example

A bank loan officer wants to analyze the data in order to know which customers (loan applicants) are risky or which are safe. The classification process includes two steps.

- Build a model for classifying customers
- Use the test model for predicting if a potential customer is risky or not

To complete these two steps, the available data are randomly separated in two groups: Training data, testing data. Usually, around 70-90% of the full data are randomly assigned to the training data and the remaining data are assigned to the validation data. The training data (risktrain.csv) in our example has 2,360 observations and the validation data (riskval.csv) has 300 observations. The data has the following attributes (information).

age	income	numkids	numcards	storecards	loans	risk
25	15777	0	0	1	0	GOODRISK
25	32876	0	0	1	0	GOODRISK
25	34824	0	1	1	0	GOODRISK
:	:	:	:	:	:	:
38	20000	3	4	5	1	BADLOSS
26	23587	2	3	4	1	BADLOSS
50	25707	2	3	5	1	BADPROFIT

The Risk field identifies customers as GoodRisk, BadProfit, or Bad Loss.

- The GoodRisk customers never missed a payment and repaid all their loans.
- The BadProfit customers repaid most of their loan (enough so we made money) but defaulted on some of it.
- The BadLoss group is where we lost money.

1. Install packages

Install packages ("rpart") as

```
install.packages("rpart")  
install.packages("rpart.plot")
```

- Remember that we need to call the packages out before we actually run the analysis? R is not smart. You need to call the packages each time.

```
library(rpart)
library(rpart.plot)
```

- Loading the training data (risktrain.csv). You can import the data now. Remember to select the correct data format. R is not smart.
- We have the tool ready, and we have the material in. Let's cook the data. Run the decision tree function.

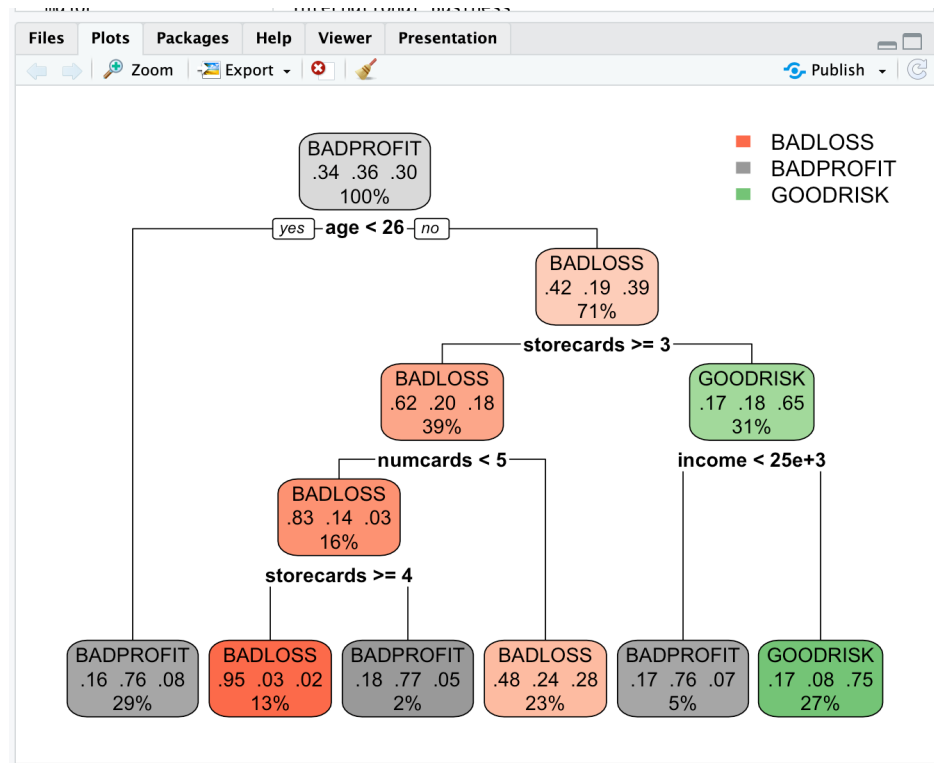
```
> names(risktrain)
[1] "age"      "income"    "numkids"   "numcards"  "storecards" "loans"     "risk"
> grep("risk", names(risktrain), ignore.case = TRUE, value = TRUE)
[1] "risk"
> tree_model <- rpart(RISK ~ ., data = risktrain, method = "class")
```

Error in eval(predvars, data, env) : object 'RISK' not found

Show Traceback
Rerun with Debug

```
> tree_model <- rpart(risk ~ ., data = risktrain, method = "class")
> rpart.plot(tree_model)
> |
```

- Although we run the analysis, we have to ask R to display the decision tree. Here type = 4 and extra = 101 are not required. The default option is type = 2, extra = "auto". You can run it again without type or extra. So you can see the difference!



6. How good is our decision tree? Let's bring the validation data in! You need to use a different dataset here (riskval.csv).

```
> riskval <- read.csv("~/Downloads/riskval (3).csv")
> names(riskval)
[1] "age"      "income"    "numkids"   "numcards"  "storecards" "loans"     "risk"
> |
```

7. Now let's predict the risk levels of the new customers from the riskval.csv file. You can name this prediction anything (not necessarily p). The new vector p (or any name of your choice) stores the predicted risk levels of the customers in the validation data. Are you wondering why you cannot see the result? Remember that R is not smart.

```
> p <- predict(tree_model, riskval, type = "class")
>
```

8. You have to kindly ask R to display this prediction result for you. Display the predicted risks:

```

Console Terminal x Background Jobs x
R 4.5.2 · ~/lab 2/
91      92      93      94      95      96      97      98      99      100
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
101      102      103      104      105      106      107      108      109      110
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
111      112      113      114      115      116      117      118      119      120
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
121      122      123      124      125      126      127      128      129      130
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
131      132      133      134      135      136      137      138      139      140
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
141      142      143      144      145      146      147      148      149      150
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
151      152      153      154      155      156      157      158      159      160
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
161      162      163      164      165      166      167      168      169      170
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
171      172      173      174      175      176      177      178      179      180
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADPROFIT
181      182      183      184      185      186      187      188      189      190
BADPROFIT BADPROFIT BADPROFIT BADPROFIT BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
191      192      193      194      195      196      197      198      199      200
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
201      202      203      204      205      206      207      208      209      210
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
211      212      213      214      215      216      217      218      219      220
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
221      222      223      224      225      226      227      228      229      230
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
231      232      233      234      235      236      237      238      239      240
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
241      242      243      244      245      246      247      248      249      250
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
251      252      253      254      255      256      257      258      259      260
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
261      262      263      264      265      266      267      268      269      270
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
271      272      273      274      275      276      277      278      279      280
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
281      282      283      284      285      286      287      288      289      290
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
291      292      293      294      295      296      297      298      299      300
BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS BADLOSS
Levels: BADLOSS BADPROFIT GOODRISK
> |

```

9. Now you want to add this prediction as a new column in the existing riskval data frame. It will be the 8th column to the data frame. Riskval is the validation data.

```
> riskval$predict <- p
>
```

10. If you want to check and see the new table, you can call it out by typing the name of the data frame, or typing *print(riskval)*. Did you see the new column?

```

Console Terminal × Background Jobs ×
R ▾ R 4.5.2 · ~/lab 2/ ↗
84 28 15880 1 1 3 1 BADPROFIT BADPROFIT
85 36 22366 1 1 1 1 BADPROFIT BADPROFIT
86 18 16570 1 2 1 1 BADPROFIT BADPROFIT
87 18 23626 2 2 3 1 BADLOSS BADPROFIT
88 27 23982 1 0 2 1 BADPROFIT BADPROFIT
89 27 15217 1 2 2 2 BADPROFIT BADPROFIT
90 18 23317 2 0 1 1 BADPROFIT BADPROFIT
91 18 23094 2 0 3 1 BADPROFIT BADPROFIT
92 26 24504 2 0 3 2 BADPROFIT BADPROFIT
93 19 22466 1 0 2 2 GOODRISK BADPROFIT
94 19 15701 2 2 3 2 BADPROFIT BADPROFIT
95 34 21582 1 2 1 2 BADPROFIT BADPROFIT
96 19 21652 2 1 2 1 GOODRISK BADPROFIT
97 26 22884 2 0 3 2 BADPROFIT BADPROFIT
98 19 21937 2 1 3 2 BADPROFIT BADPROFIT
99 19 16639 2 1 2 2 BADPROFIT BADPROFIT
100 19 23816 1 0 1 2 BADPROFIT BADPROFIT
101 19 24292 1 2 2 2 BADPROFIT BADPROFIT
102 20 15466 1 1 1 1 BADPROFIT BADPROFIT
103 20 15035 2 2 1 2 BADPROFIT BADPROFIT
104 20 18999 1 2 2 2 BADPROFIT BADPROFIT
105 20 21702 2 1 3 1 BADPROFIT BADPROFIT
106 34 23778 2 0 3 1 BADPROFIT BADPROFIT
107 32 23388 1 1 2 1 BADPROFIT BADPROFIT
108 20 16828 1 1 1 1 BADPROFIT BADPROFIT
109 20 16951 2 0 2 2 BADPROFIT BADPROFIT
110 20 24643 2 0 3 2 BADPROFIT BADPROFIT
111 36 20830 1 0 1 2 BADPROFIT BADPROFIT
112 33 18108 1 2 1 1 BADPROFIT BADPROFIT
113 21 18129 1 1 1 2 BADPROFIT BADPROFIT
114 21 23739 2 0 1 2 BADPROFIT BADPROFIT
115 21 23056 1 1 2 2 BADPROFIT BADPROFIT
116 21 24823 1 0 2 1 BADPROFIT BADPROFIT
117 21 23463 2 2 1 1 BADPROFIT BADPROFIT
118 22 16743 1 1 1 1 BADPROFIT BADPROFIT
119 22 21515 2 0 3 2 BADLOSS BADPROFIT
120 22 22100 1 0 1 2 BADPROFIT BADPROFIT
121 22 22602 2 0 2 1 BADPROFIT BADPROFIT
122 22 24479 1 2 2 1 BADPROFIT BADPROFIT
123 22 24006 1 2 1 1 BADPROFIT BADPROFIT
124 23 22845 1 2 2 1 BADPROFIT BADPROFIT
125 23 18502 1 1 1 2 BADPROFIT BADPROFIT
[ reached 'max' / getOption("max.print") -- omitted 175 rows ]
> |

```

11. Now I just want to see the actual risk value and the predicted risk value together. Which columns they are in?

Console Terminal × Background Jobs ×

R 4.5.2 · ~/lab 2/ ↗

[reached 'max' / getOption("max.print") -- omitted 175 rows]

> riskval[,7:8]

	risk	predict
1	GOODRISK	BADPROFIT
2	GOODRISK	BADPROFIT
3	GOODRISK	BADPROFIT
4	GOODRISK	BADPROFIT
5	GOODRISK	GOODRISK
6	GOODRISK	GOODRISK
7	BADLOSS	GOODRISK
8	GOODRISK	GOODRISK
9	GOODRISK	GOODRISK
10	GOODRISK	GOODRISK
11	GOODRISK	GOODRISK
12	GOODRISK	GOODRISK
13	GOODRISK	GOODRISK
14	BADLOSS	GOODRISK
15	BADLOSS	GOODRISK
16	GOODRISK	GOODRISK
17	GOODRISK	GOODRISK
18	GOODRISK	GOODRISK
19	GOODRISK	GOODRISK
20	GOODRISK	GOODRISK
21	GOODRISK	GOODRISK
22	GOODRISK	GOODRISK
23	GOODRISK	GOODRISK
24	GOODRISK	GOODRISK
25	GOODRISK	GOODRISK
26	GOODRISK	GOODRISK
27	GOODRISK	GOODRISK
28	GOODRISK	GOODRISK
29	GOODRISK	GOODRISK
30	GOODRISK	GOODRISK
31	GOODRISK	GOODRISK
32	GOODRISK	GOODRISK
33	GOODRISK	GOODRISK
34	GOODRISK	GOODRISK
35	GOODRISK	GOODRISK
36	GOODRISK	GOODRISK
37	GOODRISK	GOODRISK
38	GOODRISK	GOODRISK
39	GOODRISK	GOODRISK
40	GOODRISK	GOODRISK
41	GOODRISK	GOODRISK
42	GOODRISK	GOODRISK

12. We can use contingency table to see the two risk values side by side.

```
table (riskval[,7:8])
```

Q1: Please screenshot your codes and your output.

```
> table (riskval[,7:8])
```

	predict		
risk	BADLOSS	BADPROFIT	GOODRISK
BADLOSS	103	6	7
BADPROFIT	8	95	2
GOODRISK	5	7	67

```
>
```

Q2: How accurate is the overall prediction?

2.1. Show your calculation.

```
> sum(diag(table(riskval$risk, riskval$predict))) / sum(table(riskval$risk, riskval$predict))
```

```
[1] 0.8833333
```

2.2. Report the overall accuracy percentage.

$0.8833333 * 100 = 88.33333\%$

2.3. Interpret what this number means in practical terms.

The decision tree correctly classified about 88.33% of the customers in the validation dataset. This means that approximately 88 out of every 100 customers were placed into the correct risk category (GoodRisk, BadProfit, or BadLoss). Overall, the model performs well and makes relatively few classification errors.

Q3: Evaluate the model:

3.1. Does the model perform equally well for all three classes (GoodRisk, BadProfit, BadLoss)?

BADLOSS: 103 correct out of 116

→ $103 / 116 \approx 88.8\%$

BADPROFIT: 95 correct out of 105

→ $95 / 105 \approx 90.5\%$

GOODRISK: 67 correct out of 79

→ $67 / 79 \approx 84.8\%$

3.2. Which class has the highest misclassification?

BADLOSS errors: $6 + 7 = 13$

BADPROFIT errors: $8 + 2 = 10$

GOODRISK errors: $5 + 7 = 12$

The highest number of misclassifications is BADLOSS with 13 errors.

3.3. Why might that happen?

This may happen because some customers in the BadLoss category share similar characteristics with other groups, making it difficult for the decision tree to perfectly separate them.

3.4. If this were a real bank, what would be the financial risk of misclassifying BadLoss customers?

If BadLoss customers are misclassified as lower-risk customers, the bank may approve loans that result in financial losses. This could lead to direct monetary loss and increased credit risk for the bank.